

HW/SW CO-DESIGN

Final Project Report

Performance analysis, software optimization and a hardware acceleration proposal for two
pyperformance benchmarks

nbody · n-body gravitational simulation

pyflate · pure-Python bzip2 decompressor

Almog As 313552952

Tal Schnieder 211628656

Course 00460882 · Technion — Israel Institute of Technology

Repository https://github.com/imalmogit/hswProject_repo.git

Contents

1	Executive summary	3
1.1	Measured results	3
1.2	The main findings	4
2	Measurement method	5
2.1	Environment	5
2.2	Fixed-work measurement instead of the guide's command	5
2.3	The debug-build correction	6
3	Benchmark 1 nbody	7
3.1	What the benchmark does	7
3.2	Baseline	7
3.3	Profile and bottlenecks	8
3.4	Optimizations	10
3.5	Amdahl scorecard	13
4	Benchmark 2 pyflate	14
4.1	What the benchmark does	14
4.2	Baseline	14
4.3	Profile and bottlenecks	14
4.4	Optimizations	16
4.5	Amdahl scorecard	18
5	Hardware acceleration proposal the Move to Front unit	19
5.1	Why this component	19
5.2	Internal logic and operation	20
5.3	How the hardware module interacts with the existing software	22
5.4	Performance, area and power trade-offs	23
5.5	Expected speedup and its Amdahl limit	24
5.6	Verification	25
6	Secondary proposal Typed Float Unit for nbody	26
6.1	Concept	26
6.2	Performance bound	26
6.3	Trade-offs and failure modes	27
7	Conclusions	28
7.1	nbody	28
7.2	pyflate	28
7.3	The hardware finding	28
A	Appendix- repository and reproduction	29

1 Executive summary

We analysed two pyperformance benchmarks on the course QEMU guest, optimized both in software, and used the results to propose a hardware accelerator. Every reported measurement is reproducible from the repository linked on the title page.

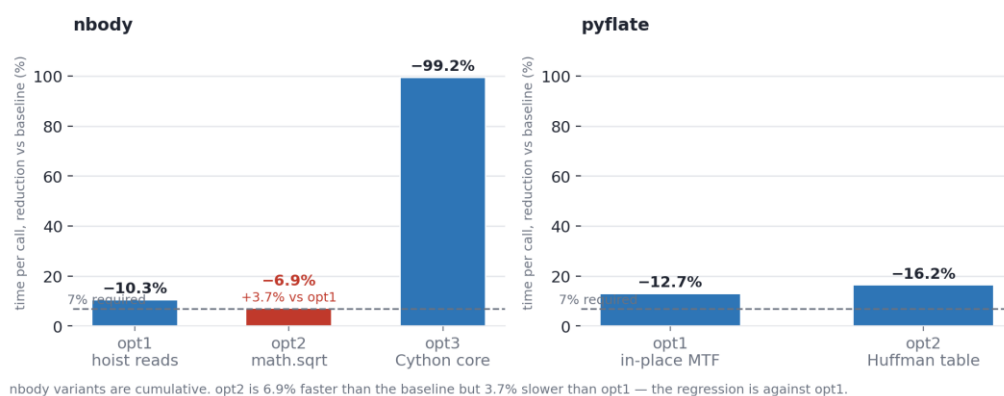
1.1 Measured results

Variant	Time / call	vs. base	Instr. / call	vs. base	IPC
Nbody- baseline	477 ms $\pm$ 2 ms	---	2.909 G	---	2.54
nbody opt1- hoist indexed reads	428 ms	−10.3%	2.662 G	−8.5%	2.58
nbody opt2- math.sqrt for pow	444 ms	−6.9%	2.765 G	−4.9%	2.59
nbody opt3- Cython typed core	3.82 ms	−99.2%	0.0197 G	−99.3%	1.86
Pyflate- baseline	3.14 s $\pm$ 0.02 s	---	16.610 G	---	2.20
pyflate opt1- in-place MTF	2.74 s	−12.7%	14.686 G	−11.6%	2.24
pyflate opt2- + Huffman table	2.63 s	−16.2%	13.943 G	−16.1%	2.20

The project requires at least a 7% improvement on two benchmarks. nbody opt1 gives 10.3% and opt3 gives 99.2%. pyflate opt2 gives 16.2%. The nbody variants are cumulative, so opt2 is 6.9% faster than the baseline but 3.7% slower than opt1. We kept opt2 because the regression explains an important limitation of software and hardware optimization: a cheaper operation does not help if reaching it costs more than the work it replaces.

Figure 1. Time-per-call reduction against each benchmark's own baseline. Both benchmarks clear the 7% threshold.

Measured improvement against each benchmark's own baseline



1.2 The main findings

nbody Interpreter overhead dominates

Only 11.1% of nbody's runtime is the floating-point arithmetic the algorithm needs. The other 88.9% is the cost of doing that arithmetic in a bytecode interpreter on heap objects.

CPython runs 539 instructions per floating-point operation, while compiled C runs 3.66. That is 147 times more.

pyflate The accelerator is only worth as much as its target. Our move-to-front unit makes its target stage $1,294\times$ faster, but the whole benchmark improves by only about $1.23\times$ because that stage is 18.5% of runtime. The whole-benchmark bound is the more useful result.

2 Measurement method

Three decisions about how we measured affect every number in this document, so we explain them here rather than in an appendix.

2.1 Environment

Host CPU	Intel Xeon E5-2630 v3 @ 2.40 GHz (Haswell)
vCPUs	1
Guest	Ubuntu 22.04.5 LTS, kernel 5.15.0-1106-kvm (QEMU/KVM)
Interpreter	CPython 3.10.12, 64-bit, --with-pydebug, CFLAGS=-g -Og
Harness	pyperformance 1.14.0, pyperf 2.10.0
Profiler	perf 5.15.200
Noise floor	roughly 2–3%. No improvement below that is claimed anywhere.

2.2 Fixed-work measurement instead of the guide's command

The project guide profiles a full pyperformance run with perf record, and we started there. That command starts about 27 processes (pip, the manager and 21 workers), and perf counts all of them. We could see this in the profile itself: it contained samples from lsb_release, dpkg-query and uname. Using pyperf's own JSON output we worked out that only 77.9% of the nbody measurement was benchmark code, which made its per-call instruction count 15.9% too high. pyflate was affected much less, because its calls are long: 94.9% benchmark code and a +1.6% over-count. We applied the same fix to both.

So every measurement here runs a single pyperf worker process that does a fixed amount of work: The fixed-worker command:

```
perf stat -r 3 -e "$G" -- \
  $PY -u "$BENCH" --worker -l 1 -w 1 -n "$N" 2>&1 | tee -a "$OUT/stat.txt" \
  || { echo "[!] perf stat failed on '$G'"; exit 1; }
```

The profiling command:

```
perf record -e cpu-clock -F 199 -g -o "$OUT/${NAME}.perf.data" -- \
  $PY -u "$BENCH" --worker -l 1 -w 1 -n "$N" \
  || { echo "[!] perf record failed"; exit 1; }
```

The flame graphs command:

```
perf record -e cpu-clock -F 99 --call-graph dwarf,16384 \
  -o "$OUT/${NAME}_dwarf.perf.data" -- \
  $PY -u "$BM/bm_${NAME}/run_benchmark.py" --worker -l 1 -w 1 -n "$N"
```

N = 100 for nbody, which gives 101 benchmark calls, and N = 20 for pyflate, which gives 21. We use the same flags for every variant, including the baselines, so every run does exactly the same amount of work and the counters can be compared directly. perf stat counts the whole process, so we divide its totals by N+1 to get a per-call figure.

Two details about the guest matter here. Its virtual PMU counts events but does not deliver sampling interrupts, so perf record without an explicit event opens the hardware cycles event, reports no error, and then records nothing at all.

That is why every profiling command names the event: `-e cpu-clock -F 199`. The flame-graph captures also use DWARF unwinding, because the interpreter is built at `-Og` and frame-pointer unwinding does not work on it.

2.3 The debug-build correction

The course interpreter is built with `--with-pydebug`. Code that only exists because of that build is 3.35% of nbody's profile and 18.31% of pyflate's. In pyflate, four of the five biggest symbols after the eval loop are debug-allocator functions. We leave this code out of every bottleneck figure, and the profile tables have a "release-equivalent" column that divides by the non-debug share (0.9665 for nbody, 0.8169 for pyflate) to estimate what a normal interpreter would show.

This correction matters for the hardware argument. Without it, pyflate would appear allocation-bound for a reason that disappears on a normal interpreter. The same applies to our measurements: 0.321 G of pyflate opt1's gain, or 17%, is debug-allocator work that a release build would never run.

3 Benchmark 1 nbody

3.1 What the benchmark does

nbody is an n-body gravitational simulation taken from the Computer Language Benchmarks Game. It models the sun and the four gas giants under Newtonian gravity. One benchmark call computes the total energy of the system, runs 20,000 timesteps, then computes the energy again.

The inner loop body runs 200,000 times per call (20,000 timesteps $\times$ 10 body pairs) and does about 27 floating-point operations each time, so roughly 5.4 million per call. There is no third-party numeric library: every value is an ordinary CPython float, so every arithmetic result is a PyFloatObject allocated on the heap. Positions and velocities are Python lists of three floats, written in place, which goes through the generic sequence protocol on every access.

3.2 Baseline

The pyperformance mean over 20 processes is 477 ms $\pm$ 2 ms. We repeated the whole measurement from scratch in a second session, on a fresh clone of the repository, and got 477 ms and 2.909 G instructions per call again. That is the same to four significant digits, on a guest whose noise floor is 1.5%.

Counter	Fixed-work run (101 calls)	Per call
task-clock	48.45 s	479.7 ms
instructions	293.8 G	2.909 G
cycles	115.5 G	1.144 G
IPC	2.54	---
branches	66.9 G (22.8% of instructions)	---
branch-miss rate	0.48%	---
cache references	11.9 M (0.040 per 1K instr.)	---
LLC misses per 1K instructions	0.0011	---
page faults	3,930	---

What the counters tell us:

nbody barely touches memory. Five bodies and their vectors fit easily in L1. The limit is how many instructions the interpreter has to run, not how much data it moves, so an accelerator aimed at memory traffic would gain nothing here. The branch numbers need some care. A 0.48% miss rate and 22.8% of instructions being branches are both normal for a bytecode interpreter, because most of those branches are the eval loop dispatching the same opcodes over and over. The low miss rate does not mean there is no dispatch cost to remove. It means the cost is issue bandwidth, not misprediction.

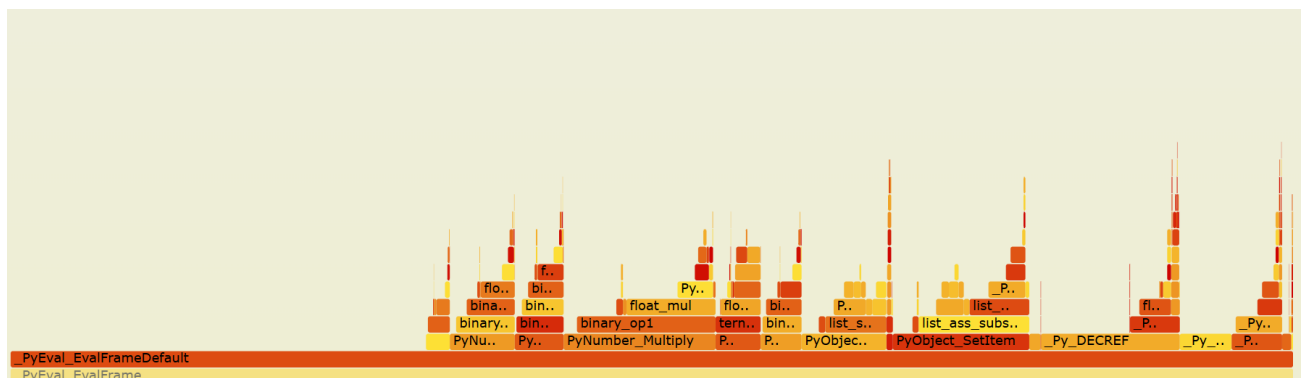
3.3 Profile and bottlenecks

9,000 cpu-clock samples at 199 Hz over 48.5 s, none lost. The flame graph resolves 462 distinct frames with no raw addresses. We grouped the symbols by mechanism. The instructions-per-call column takes each mechanism's share of the samples and applies it to the total instructions per call. That assumes IPC is roughly the same across mechanisms, so those figures are indicative rather than counted.

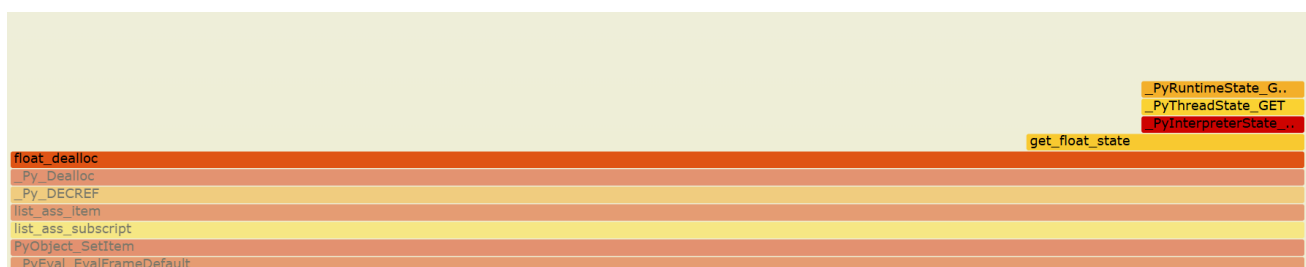
Mechanism	Profile	Release-equiv.	Instr. / call
Interpreter dispatch	45.56%	47.1%	1.325 G
Sequence element access	12.34%	12.8%	0.359 G
Float boxing and refcounting	12.13%	12.6%	0.353 G
REAL float arithmetic	10.69%	11.1%	0.311 G
Operator dispatch	10.07%	10.4%	0.293 G
Index to C integer	5.09%	5.3%	0.148 G
Debug-build only	3.35%	excluded	0.098 G
Uncategorised tail	0.70%	0.7%	0.020 G

Where the time actually goes:

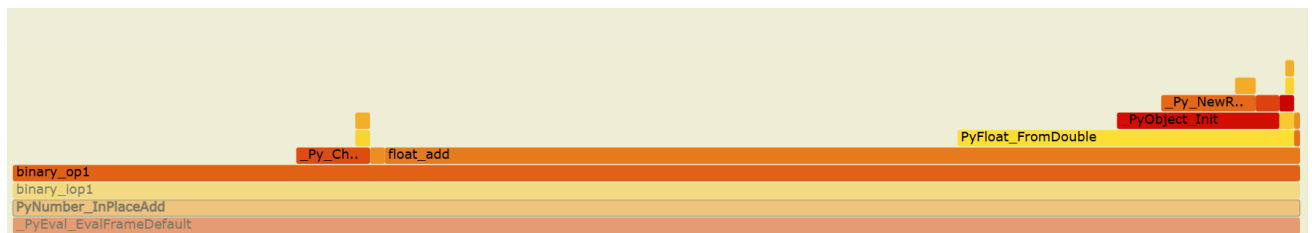
- Interpreter dispatch- `_PyEval_EvalFrameDefault` alone: bytecode fetch, decode and dispatch, and value-stack maintenance.



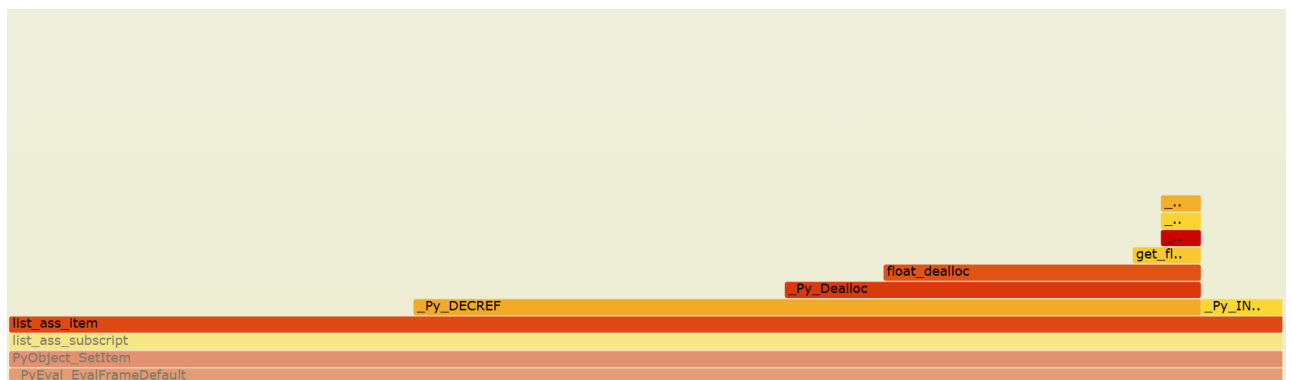
- Float boxing- `PyFloat_FromDouble`, `float_dealloc`, `_Py_NewReference`, `_Py_Dealloc`. Every intermediate value in $dx \cdot dx + dy \cdot dy + dz \cdot dz$ becomes a heap object, which is then refcounted and freed a few microseconds later. Allocating and destroying the result costs a little more than computing it.



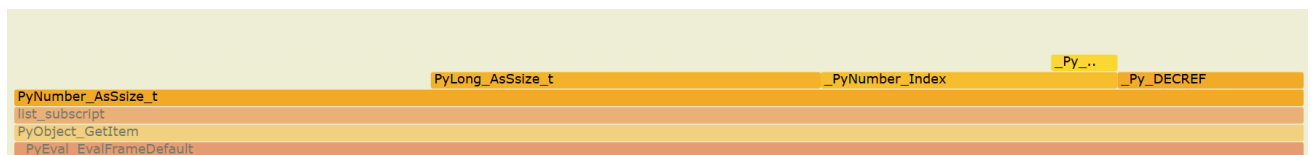
- Operator dispatch- `binary_op1`, `PyNumber_Multiply` and the rest of the `PyNumber_*` family: working out at runtime, for every single operation, which `__mul__` applies to these two objects. The types never change during the whole run.



- Sequence element access- `list_ass_item`, `list_item`, `PyObject_GetItem` and friends: reading and writing `r[0]` and `v1[2]` through the generic sequence protocol.



- Index to C integer- `PyNumber_AsSsize_t`, `_PyNumber_Index`, `PyLong_AsSsize_t`: turning the Python integer 0 into a C `ssize_t` before it can be used as an index. The indices are literal constants.



The key finding:

Only 11.1% of `nbody`'s runtime is the floating-point arithmetic the algorithm needs. The other 88.9% is what it costs to do that arithmetic in a bytecode interpreter on heap objects: about eight instructions of overhead for every instruction of real physics.

Boxing (12.6%), element access (12.8%), operator dispatch (10.4%) and index conversion (5.3%) add up to 41.1% of the runtime. All of them exist only because the values are generic heap objects reached through generic protocols. The types and shapes never change during the run- five bodies, three coordinates, all doubles- but the interpreter works this out again on every operation.

The wide interpreter frames and repeated list-access paths show that execution time is dominated by CPython overhead rather than the numerical calculation.

The interpreter spends 539 instructions on each floating-point operation. Each row below is that mechanism's share of the profile applied to the 539 and rounded to a whole number, which is why the column adds up to 537 rather than 539.

Mechanism	Instructions per float op
Interpreter dispatch	245
Sequence element access	66
Float boxing and refcounting	65
Real float arithmetic	58
Operator dispatch	54
Index to C integer	27
Debug-build only	18
Uncategorised tail	4
TOTAL	539
opt3, compiled C core	3.66

3.4 Optimizations

The variants are cumulative: opt2 includes opt1, and opt3 includes opt2. We checked that each one reproduced the original physics before taking any timing. `scripts/verify_nbody.py` compares the final total energy after 20,000 timesteps.

Opt 1 Hoist indexed reads into local variables

This targets sequence element access, 12.8% of the runtime. On every pair iteration the inner loop reads and writes `v1[0]`, `v1[1]` and `v1[2]` through the generic sequence protocol. In CPython, every indexed access requires dynamic sequence handling, index conversion, and bounds-related work.

We replaced repeated accesses with local variables:

```
v1x, v1y, v1z = v1
v2x, v2y, v2z = v2

v1[0] = v1x - dx * b2m
v1[1] = v1y - dy * b2m
v1[2] = v1z - dz * b2m
```

Verification: final energy is bit-exact with the original. Result: -10.3% time, -8.5% instructions.

Mechanism	baseline	opt1	change
Sequence element access	0.359 G	0.169 G	-0.190 G (-53%)
Index to C integer	0.148 G	0.071 G	-0.078 G (-52%)
Interpreter dispatch	1.325 G	1.396 G	+0.071 G

Mechanism	baseline	opt1	change
Net of these three			-0.197 G

The measured difference is -0.247 G per call, so these three mechanisms explain 80% of it. We did not plan the drop in index conversion: fewer subscripts simply means fewer Python integers unwrapped into C ssize_t. Dispatch went up because the unpack adds bytecodes, but cheaper ones. Time fell by 10.3% against 8.5% fewer instructions because IPC also improved, from 2.54 to 2.58.

Opt 2 Replace generic pow with math sqrt

The original gravitational magnitude calculation used Python's generic exponentiation operator:

```
mag = dt * ((dx * dx + dy * dy + dz * dz) ** (-1.5))
```

We rewrote it as:

```
d2 = dx * dx + dy * dy + dz * dz
mag = dt / (d2 * sqrt(d2))
```

The two forms are mathematically equal for $d2 > 0$. The final energy differs by 9.5e-15 relative, which is the last-ulp difference we expected from reordering the operations. In opt1 the whole pow path (`_ieee754_pow_fma`, `float_pow` and `libm's pow`) is 3.2% of the runtime, and that is the entire budget this change has to work with.

opt2 is 3.7% slower than opt1 and runs 103 M more instructions per call. opt2 is opt1 plus one change, so the comparison isolates that change.

Mechanism	opt1	opt2	change
Real float arithmetic	0.303 G	0.244 G	-0.059 G ← the win
math.sqrt call path	0.024 G	0.104 G	+0.080 G
Interpreter dispatch	1.396 G	1.432 G	+0.036 G
Debug-build only	0.077 G	0.095 G	+0.018 G
Index to C integer	0.071 G	0.086 G	+0.016 G
Sequence element access	0.169 G	0.181 G	+0.012 G
Operator dispatch	0.268 G	0.272 G	+0.004 G
Float boxing	0.352 G	0.348 G	-0.004 G
NET			+0.103 G

The measured difference is +0.103 G, so the table accounts for the change to within 0.3%. The symbols confirm what happened. In opt1 the pow path is 3.15% of the runtime; in opt2 all three of its symbols are gone, replaced by `__sqrt_finite` (0.08%) and `__sqrt` (0.01%). The arithmetic did get cheaper, by 0.059 G, which is two thirds of the 3.2% budget. The program still got slower, because writing the cheaper formula in Python costs a function call, the math module's dispatch, a global name lookup, unboxing

the argument and two extra boxed temporaries on every iteration. That is 0.166 G spent to save 0.063 G, or about 515 extra instructions per pair iteration.

Why we kept this negative result: the arithmetic was never the bottleneck. Making it cheaper saved 0.059 G instructions, while expressing the new formula in Python cost 0.166 G. On a release build that cost would still be 0.148 G, more than twice the saving.

This is also a design constraint on the hardware proposal: the path to an accelerator must cost less than the work it replaces. We measured that limit directly in opt2.

Opt 3 Cython typed core and measurement correction

We moved the whole `advance()` loop into a typed Cython function that works on C doubles held in fixed C arrays. `report_energy()` and `offset_momentum()` stay in Python, so only the hot loop is compiled. The pair iteration order is the same as `combinations()` in the original, so the floating-point values accumulate in the same order. Verification: the final energy matches opt2 exactly.

opt3 was fast enough to break our own measurement. The worker reported 2.736 G instructions over 101 calls, which divides to 27.1 M per call. But the benchmark's own time ($101 \times 3.82 \text{ ms} = 386 \text{ ms}$) against the 617 ms of task-clock the process used showed that 37% of the measurement was interpreter startup, not the benchmark. This is the same problem we removed in section 2.2, back again because the benchmark got two orders of magnitude smaller.

The fix is to measure the slope instead of assuming the intercept is zero. We ran the same worker at three values of N and fitted instructions against call count using the two smaller points, keeping the third back as a check:

calls	instructions	task-clock
21	1,157,153,533	308.56 ms
61	1,947,033,411	464.37 ms
101	2,735,572,499	617.02 ms ← held out
startup (intercept)	742.5 M instructions, 226.8 ms	
per call (slope)	19.747 M instructions, 3.895 ms	
naive total ÷ 101	27.085 M, inflated by +37.2%	
prediction error on the held-out point	+0.05% instructions, +0.51% time	
slope time vs. independent pyperf mean	3.895 vs. 3.82 ms- 2.0% apart	

That table contains two independent checks. The fit predicts a point it was not fitted on to within 0.05%, and its time slope agrees to within 2% with the pyperf mean, which comes from 20 separate processes measured a completely different way. The corrected figures are 0.0197 G instructions and 3.82 ms per call: 147× fewer instructions and 125× faster than the baseline. To put that in scale, the baseline's float-arithmetic bucket on its own is 0.311 G per call, 16× more than everything opt3 does in a call. Even the part that looked like real computation was about 94% object handling.

```

===== bm_nbody =====
nbody: Mean +- std dev: 477 ms +- 2 ms
      293810593271      instructions      #      2.54  insn per cycle      ( +- 0.00% )

===== bm_nbody_opt1 =====
nbody_opt1: Mean +- std dev: 428 ms +- 2 ms
            268837839048      instructions      #      2.58  insn per cycle      ( +- 0.01% )

===== bm_nbody_opt2 =====
nbody_opt2: Mean +- std dev: 444 ms +- 2 ms
            279241990931      instructions      #      2.59  insn per cycle      ( +- 0.07% )

===== bm_nbody_opt3_cython =====
nbody_opt3_cython: Mean +- std dev: 3.81 ms +- 0.01 ms
                  2735649442      instructions      #      1.86  insn per cycle      ( +- 0.02% )

```

3.5 Amdahl scorecard

The ceilings are release-equivalent shares: we take out the debug-only bucket before working out the fraction.

Optimization	Targeted	Ceiling	Measured	Verdict
opt1	element access + index conversion	18.1%	-8.5% instr, -10.3% time	held
opt2	the generic pow path	3.2%	arithmetic fell 2.2%, net +3.7% slower	2/3 of ceiling; negative result
opt3	boxing, dispatch, indexing	41.1% (1.70×)	-99.3% instr, 125× faster	far exceeded

Amdahl's law tells you what happens when the part you target goes to zero and nothing else changes. Cython changes everything else as well. It also removes interpreter dispatch, which is 47.1% of the baseline and something the hardware proposal does not touch, and it makes the remaining arithmetic natively fast, because a C double multiply is one instruction while CPython's float_mul wraps allocation, refcounting and type dispatch around the same operation. The point for the hardware argument is that an accelerator which removes only boxing and dispatch, and leaves the bytecode loop in place, really is limited to 1.70×.

4 Benchmark 2 pyflate

4.1 What the benchmark does

pyflate is a stand-alone DEFLATE and bzip2 decompressor written in pure Python. The pyperformance benchmark decompresses the bundled file `data/interpreter.tar.bz2`, 67,562 bytes in and 399,360 bytes out, and checks the result against a fixed MD5 digest. That check was useful to us: any optimization that breaks the decode fails straight away, instead of quietly giving a faster wrong answer.

The data file is bzip2, so only `bzip2_main` runs and the whole DEFLATE half of the module is dead code here. The decode has four stages: Huffman decode, move-to-front, the inverse Burrows-Wheeler transform, and a final run-length pass. The first two run once per decoded symbol and dominate the profile. On this file we measured 89,837 move-to-front operations on a 145-entry table.

The move-to-front table, called `favourites`, is a Python list of single-byte objects, up to 256 entries and 145 for this file. That choice of data structure is what our hardware proposal targets.

4.2 Baseline

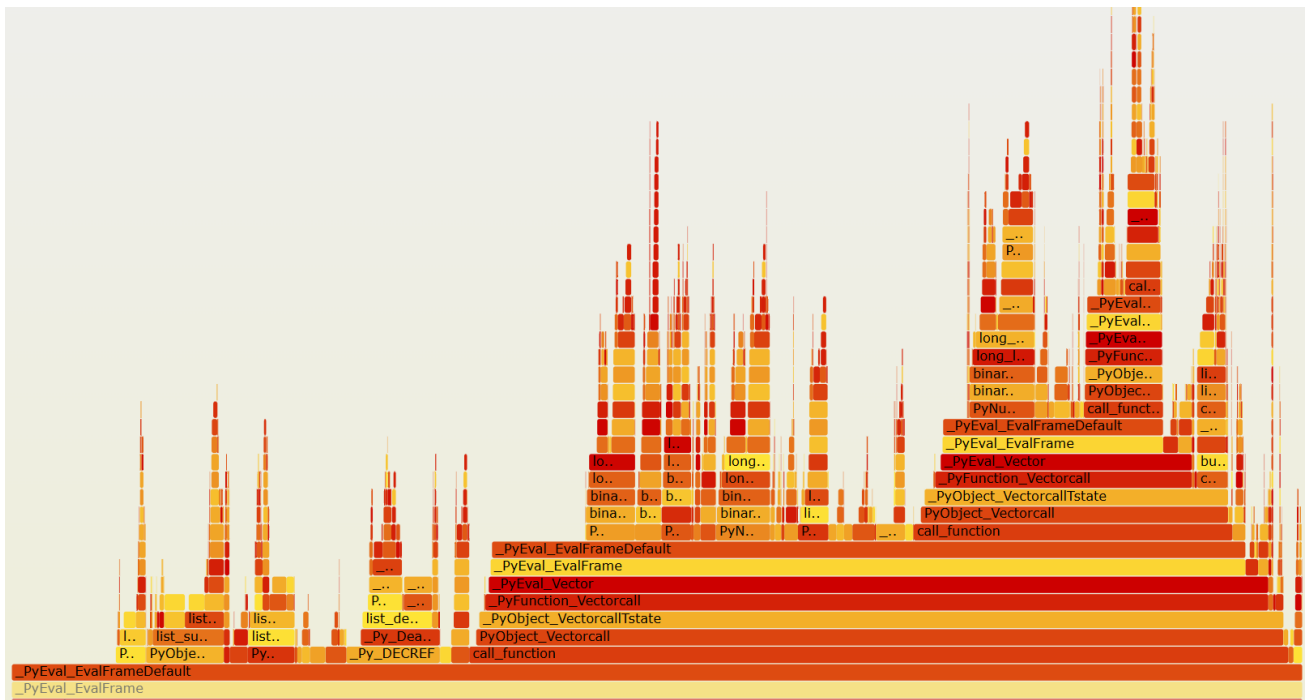
The pyperformance mean over 20 processes is $3.14 \text{ s} \pm 0.02 \text{ s}$. We repeated the whole measurement from scratch in a second session on a fresh clone. The mean came back identical at 3.14 s, and the per-call instruction count was within 0.8% (16.610 G against 16.743 G), on a guest whose noise floor is 2.5%.

Counter	pyflate	nbody, for contrast
instructions per call	16.610 G	2.909 G
IPC	2.20	2.54
cache references per 1K instr.	0.491	0.040
cache-miss rate	7.79%	2.78%
LLC misses per 1K instructions	0.0381	0.0011
page faults (fixed-work run)	127,912	3,930

pyflate has the opposite shape to nbody. It issues $12\times$ more cache references per instruction and misses $2.8\times$ more often, which gives $35\times$ the last-level miss rate, and its 127,912 page faults show the same thing from the allocator's side. These numbers are why we treat pyflate as a data-movement problem and nbody as a compute problem, and why an accelerator that removes object churn is the right kind of proposal here.

4.3 Profile and bottlenecks

13,000 cpu-clock samples at 199 Hz over 66.6 s, none lost.



MTF calls per benchmark call	89,837
Table size in the test stream	145 entries
Instructions per MTF operation	$\approx 21,400$
Time in the MTF stage	368 ms of 3,140 ms (release-equivalent)

We measured the instructions-per-operation figure rather than estimating it. `opt1` changes `move_to_front` and nothing else, and it removes 1.924 G instructions per benchmark call. Divided by 89,837 operations, that is 21,415 instructions per move-to-front. It is a lower bound on the original cost, because the in-place version still does a `pop` and an `insert`.

4.4 Optimizations

The variants are cumulative. We checked that each one reproduced the benchmark's own MD5 digest before taking any timing.

Opt 1 In place Move to Front

The two versions do the same thing: both move the element at index c to position 0 and keep the order of everything else. The new one replaces five list allocations and a full rebuild with two memmoves inside a single list object, and allocates nothing. No external library is used. This is the software version of what our hardware proposal does. Result: -12.7% time, -11.6% instructions. The original implementation rebuilt the entire list on every operation:

```
def move_to_front(l, c):  
    l[:] = l[c:c + 1] + l[0:c] + l[c + 1:]
```

We replaced it with an in-place implementation:

```
def move_to_front(l, c):  
    l.insert(0, l.pop(c))
```

Mechanism	baseline	opt1	change
List / slice (MTF)	1.590 G	0.447 G	−1.143 G (−72%)
Debug-build-only code	3.041 G	2.720 G	−0.321 G
Dict lookup	0.641 G	0.458 G	−0.183 G
Function call machinery	1.661 G	1.570 G	−0.091 G
Memory allocator	0.924 G	0.840 G	−0.084 G
Integer and bit operations	0.696 G	0.745 G	+0.049 G
Net of these six			−1.773 G

The measured difference is −1.924 G per call, so these six explain 92% of it. We removed 72% of the move-to-front cost, which is exactly the mechanism we targeted: `list_ass_slice` falls from 2.09% to 0.05%, `list_concat` and `list_slice` disappear, and `list_dealloc` falls from 2.41% to 0.25%. What is left is `list_item` at 1.32%, and that is the table read the accelerator would replace. One caveat: 0.321 G of the gain is debug-allocator work that a release interpreter does not run, so 17% of this improvement would not show up on a normal Python.

Opt 2 Table driven Huffman decode

`find_next_symbol` scans the symbol table one entry at a time, calling `snoopbits` for each candidate and reading three Python attributes per entry. `self.table` is already sorted by (bits, code), so grouping it by code length keeps the same ascending-length search order while replacing the scan with one dictionary lookup per distinct length. Result: −16.2% cumulatively, or a further −4.0% against `opt1`.

Mechanism	opt1	opt2	change
Interpreter dispatch	3.936 G	3.249 G	−0.687 G
Integer and bit operations	0.745 G	0.622 G	−0.123 G
Tuple access	0.283 G	0.243 G	−0.041 G
Operator dispatch	0.386 G	0.353 G	−0.033 G
Dict lookup (Huffman)	0.458 G	0.467 G	+0.009 G
Function call machinery	1.570 G	1.754 G	+0.184 G
Net of these six			−0.691 G

We expected this to reduce the cost of dict lookups. It did not. Dict lookup went up slightly, by 0.009 G, and the call machinery went up by 0.184 G, because `dict.get()` is a method call where the old code was an inline comparison. The real gain came from removing the interpreted loop: 0.687 G of the 0.743 G

saved, 92% of the improvement, is interpreter dispatch. A dictionary `get()` removes the loop itself, not just the comparisons inside it, and paying 0.184 G of extra call machinery for that is worth it. Our prediction was right about the direction and wrong about the mechanism, and the profile is what showed us. We report it instead of quietly correcting it, because an attribution we checked is worth more than one we did not.

```

===== bm_pyflate =====
pyflate: Mean +- std dev: 3.14 sec +- 0.02 sec
          348809459926      instructions      #      2.20  insn per cycle      ( +-  0.07% )

===== bm_pyflate_opt1 =====
pyflate_opt1: Mean +- std dev: 2.74 sec +- 0.04 sec
              308409917286      instructions      #      2.24  insn per cycle      ( +-  0.15% )

===== bm_pyflate_opt2 =====
pyflate_opt2: Mean +- std dev: 2.63 sec +- 0.02 sec
              292812563603      instructions      #      2.20  insn per cycle      ( +-  0.22% )

```

4.5 Amdahl scorecard

Optimization	Targeted	Ceiling	Measured	Verdict
opt1	MTF list ops + allocator	18.5%	−11.6% instr, −12.7% time	held; reached 69% of the ceiling
opt2	Huffman dict lookup	4.7% (1.05×)	−5.1% instr vs. opt1	right size, wrong mechanism

5 Hardware acceleration proposal the Move to Front unit

RTL	hardware/mtf_unit.v (Verilog-2005)
Testbench	hardware/tb_mtf_unit.v (self-checking)
Stimulus generator	hardware/gen_stimulus.py- captured from a real bzip2 decode
Build and run	cd hardware && make
Full specification	hardware/README.md
Verification	PASS — 89,837 operations, output stream identical to pyflate

5.1 Why this component

- Fixed, tiny working set: 256 bytes, resident in the unit. No memory traffic, no cache pressure, no DMA.
- Very regular: one index in, one byte out, one well-defined change to the table. No floating point, no variable latency, no exceptions.
- Runs constantly: 89,837 times per benchmark call, and in bzip2 generally once per decoded symbol.
- Already proven in software: opt1 removed 72% of the stage's cost and made the whole benchmark 12.7% faster. The hardware removes the rest.
- Not specific to this benchmark: move-to-front is a standard transform in bzip2, in some LZ77 variants, and in certain cache-replacement policies.

Today the operation costs about 21,400 instructions to move one byte to the front of a 145-entry list. The unit does it in one cycle.

5.2 Internal logic and operation

The unit is a table held in flip-flops, a read multiplexer, a conditional shift network and a two-state controller. One promotion completes per clock cycle.

mtf_unit — internal datapath and control

Latency 1 cycle · throughput 1 operation per cycle, no bubbles · target 200 MHz single-cycle

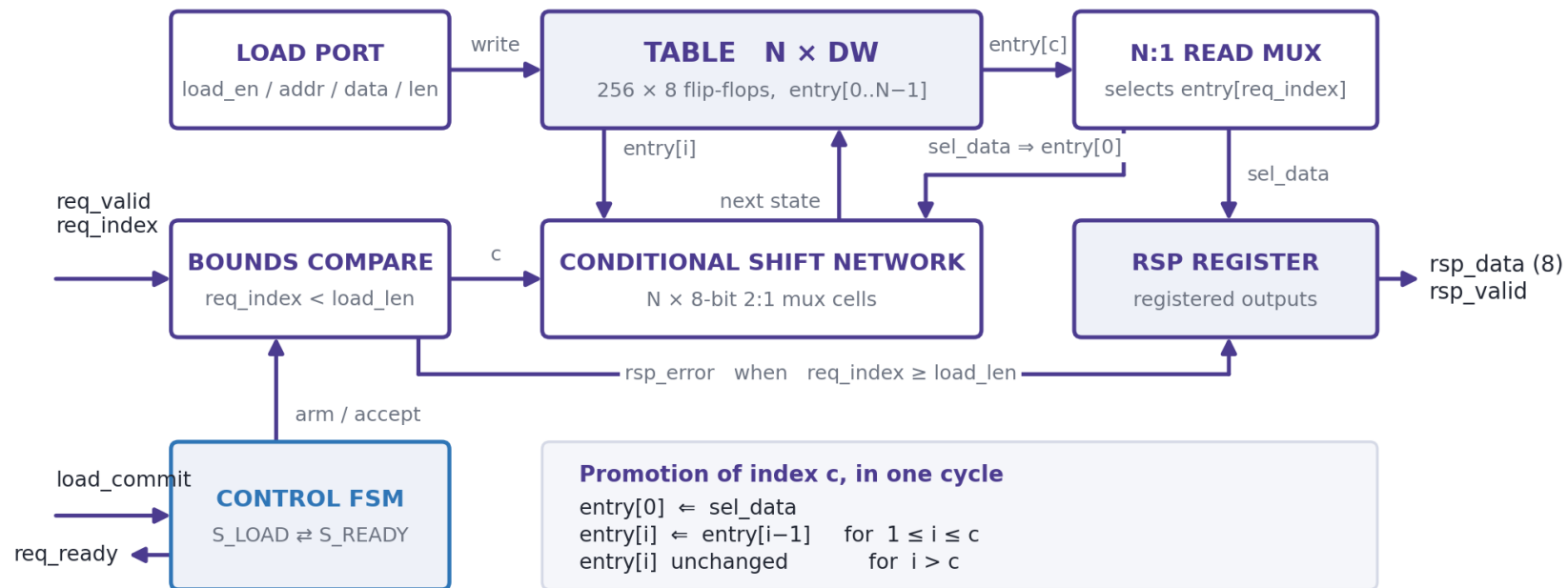


Table storage 2,048 flip-flops · shift network 2,048 8-bit 2:1 mux cells · one 256:1 read mux · one 9-bit comparator · 1-bit FSM state

Datapath three elements

An N:1 read multiplexer selects entry[req_index]. That value, sel_data, is both the answer returned to software and the value written back into position 0. The promotion itself is done by a conditional shift network of N 8-bit 2:1 multiplexers: each entry either keeps its current value or takes its neighbour's, decided by one comparison of its own position against req_index. A 9-bit magnitude comparator does the bounds check against the latched table length.

```
entry[0]      <= sel_data
entry[i], 1 <= i <= c  <= entry[i-1]
entry[i], i > c      <= entry[i]    (unchanged)
```

Every entry updates in the same cycle from its own 2:1 multiplexer, so the promotion is not a loop that shifts one element at a time. It is a single parallel register transfer. That is the whole reason the unit takes one cycle where the Python version takes about 21,400 instructions: software has to walk the list, and hardware does not.

Control two states

S_LOAD accepts table writes and holds req_ready low. load_commit latches the length and moves the unit to S_READY, where it accepts one request per cycle. Another load_commit takes it back to S_LOAD for the next bzip2 block, because bzip2 builds a new table for each block. The response strobes are low by default and only go high on an accepted request, so an out-of-range index raises rsp_error and changes nothing.

Interface

Parameters: N = 256 entries, DW = 8 symbol bits, IW = 8 index bits. All signals are synchronous to clk with an asynchronous active-low reset rst_n.

Signal	Dir	Width	Description
clk	in	1	clock
rst_n	in	1	asynchronous active-low reset
load_en	in	1	write one table entry
load_addr	in	8	entry address
load_data	in	8	entry value
load_len	in	9	number of valid entries, 0...256
load_commit	in	1	latch load_len, arm the table
req_valid	in	1	request a promotion
req_index	in	8	index c to promote
req_ready	out	1	high when the table is armed
rsp_valid	out	1	response strobe

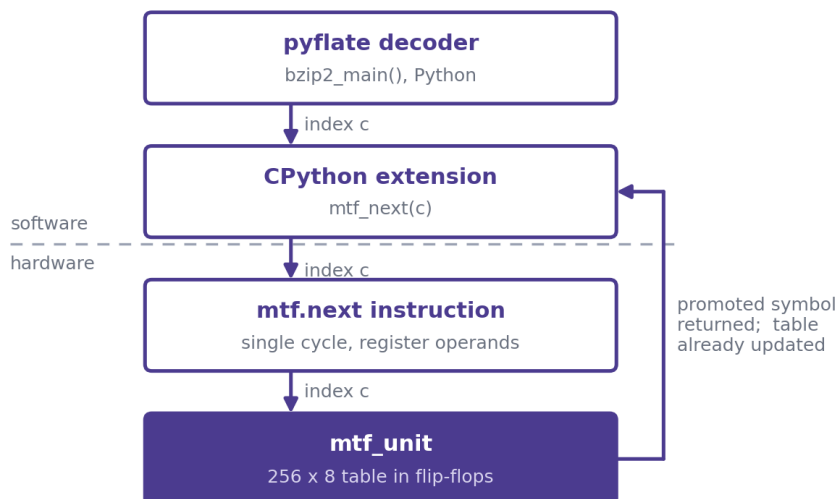
Signal	Dir	Width	Description
rsp_data	out	8	the promoted symbol
rsp_error	out	1	req_index $\geq$ load_len

Latency	1 cycle (registered outputs)
Throughput	1 operation per cycle, back to back, no bubbles
Stall conditions	none once armed
Target frequency	200 MHz single-cycle

5.3 How the hardware module interacts with the existing software

How the unit reaches the existing software

move_to_front() becomes one call into a CPython extension module.
The bzip2 decoder itself is unchanged.



Cost of one promotion

- today, pure Python ~ 21,400 instructions, ~ 4.1 us
- Option A, custom instruction 1 cycle, ~ 5 ns
- Option B, memory-mapped ~ 100-200 ns

Option A ISA extension

Three custom instructions, in the style of RISC-V custom-0. The unit sits next to the integer ALU and writes back through the normal result path. mtf.next takes one cycle and has no memory operand.

Instruction	Operands	Effect
mtf.load rs1, rs2	rs1 = index, rs2 = value	write one table entry
mtf.commit rs1	rs1 = length	arm the table
mtf.next rd, rs1	rs1 = index	rd $\leftarrow$ symbol; table updated in place

On the software side we would add a CPython extension module with two functions, `mtf_load(table)` and `mtf_next(c)`, and replace `move_to_front` with a call to `mtf_next`. Nothing else in the decoder changes. The table lives in the unit, the promotion happens as a side effect of the read, and Python never touches the list again.

Option B Memory mapped peripheral

A 32-bit AXI4-Lite slave with six registers, plus a small kernel driver or a `/dev/mem` mapping and the same extension module. No DMA: one index in and one byte out per operation is far below the point where setting up a DMA transfer would pay for itself.

Offset	Access	Register
0x00	W	CTRL- bit 0 commit, bit 1 soft_reset
0x04	W	LEN- table length, bits 8:0
0x08	W	LOAD- bits 15:8 address, bits 7:0 data
0x0C	W	REQ- index; the write issues the operation
0x10	R	RSP- bit 8 error, bits 7:0 promoted symbol
0x14	R	STATUS- bit 0 ready

Why the choice matters

In the nbody half of this project, `opt2` replaced `pow` with a cheaper formula and made that benchmark 3.7% slower, because getting to the cheaper operation cost a Python function call. The same thing can happen to any accelerator, so we costed the path to the unit explicitly:

Path	Cost per operation
Today, pure Python	$\approx 21,400$ instructions $\approx 4.1 \mu\text{s}$
Option A — custom instruction	1 cycle $\approx 5 \text{ ns}$, plus the Python call
Option B — two bus round trips	$\approx 100\text{--}200 \text{ ns}$, plus the Python call

Both are far smaller than the $4.1 \mu\text{s}$ they replace, so either option works here. The general rule still holds, and we measured it once in this project: if the path to an accelerator costs more than the work it replaces, the accelerator loses.

5.4 Performance, area and power trade-offs

Frequency

The critical path is the N:1 read multiplexer followed by the write into the shift network, which is about eight levels of 2:1 multiplexing plus setup. A 200 MHz single-cycle target is conservative on any modern process. If a higher clock were needed, putting a register on `sel_data` would split the path into two stages. Latency would become 2 cycles and throughput would stay at one operation per cycle, and the decoder does not care about latency because it uses each symbol before it asks for the next.

Area

Element	Cost
Table storage	$256 \times 8 = 2,048$ flip-flops
Shift network	2,048 8-bit-wide 2:1 mux cells
Read multiplexer	256:1, 8 bits wide
Bounds comparator	9-bit
Control	1-bit FSM state + 9-bit length register

That is roughly 2 K flip-flops and 2 K mux cells. It is small next to a 32 KB L1 cache, and it is one shared block rather than something that has to be repeated in every core.

Power and energy

The unit is clock-gated outside S_READY. On each operation only the entries below req_index switch, so the average switching depends on the mean promoted index rather than on the size of the whole table. For a move-to-front table that index is much smaller than N, because the whole point of the transform is to keep recently used symbols near the front.

Peak power goes up while the unit is working, so the claim we can defend is about energy per operation, not peak power. Replacing about 21,400 executed instructions, five heap allocations and around 1,450 refcounted pointer copies with one register transfer cuts the energy of that operation by orders of magnitude, even if the instantaneous power is higher.

5.5 Expected speedup and its Amdahl limit

There are two reasonable ways to define "the stage", and they give different bounds, so we give both instead of picking the one that looks better. The narrow definition is the move-to-front list operations on their own. The wide one also includes the allocator work those slices cause, which the accelerator removes as well, because the allocations stop happening altogether.

Basis	Narrow	Wide
Share of profile	9.57%	15.13%
Share excluding debug-build-only code	11.7%	18.5%
Time per benchmark call	368 ms	581 ms
Hardware time for all 89,837 operations	449 μ s	449 μ s
Speedup on the stage	820×	1,294×
AMDAHL CEILING, whole benchmark	1.13×	1.23×

At 200 MHz the unit performs all 89,837 operations of a decode in 449 μ s, compared with 368–581 ms in software. That makes the stage effectively negligible, yet the whole benchmark still improves by only about 1.23×

This is Amdahl's law. The $1.23\times$ whole-benchmark bound is more informative than the $1,294\times$ stage-only figure. For comparison, the software fix alone, opt1, already gave -12.7% time and -11.6% instructions.

The assumptions behind these numbers, so they can be checked:

- The table fits in $N = 256$ entries. bzip2 guarantees this, and if it were ever broken the unit raises `rsp_error` instead of misbehaving.
- There is one MTF operation per decoded symbol, which our captured trace confirms.
- The path to the unit is cheap compared with $4.1\ \mu\text{s}$, which is true for both options above.
- The profile share comes from a `--with-pydebug` interpreter. $0.321\ \text{G}$ of opt1's gain is debug-only work that a release build does not do, so the benefit on a release build is smaller.

5.6 Verification

`gen_stimulus.py` hooks `move_to_front` during a real decode of `interpreter.tar.bz2` and records the starting table plus every (index, promoted symbol) pair that Python produced. `pyflate` calls `move_to_front` from two places, a 6-entry selector table and the 145-entry symbol table, and the generator picks the busier one. The testbench loads the captured table, sends all 89,837 requests back to back with no gaps, and compares every response against the value Python produced.

```
reset      : req_ready low until the table is armed
arm        : req_ready high after load_commit
bounds check: index 145 correctly rejected
issued     : 89837
checked    : 89837
mismatches : 0
RESULT: PASS -- output stream matches pyflate exactly
```

Matching the Python output symbol for symbol is the check that matters. It shows the unit is a drop-in replacement for the function it removes, and not just a piece of logic that looks reasonable.

6 Secondary proposal Typed Float Unit for nbody

The accelerator we implemented targets pyflate, because that operation is small enough to specify and verify completely in RTL. The nbody proposal below is an analysis rather than RTL. We include it because its bound is the main co-design finding of the project.

6.1 Concept

On each of the 5.4 M operations per call, the interpreter works out the same facts again: that both operands are doubles, which `__mul__` applies, that the index literal is a small integer, and that the result has to become a heap object. The unit caches the resolved pair of operand types for each bytecode site. A site id is just the bytecode offset inside a code object, and that is what makes the cache work, because the same site sees the same types every time a numeric loop runs. On a hit the unit computes on raw doubles held in its own register file, and boxes a value only when it escapes to Python.

We add two new opcodes, `FPU_OP` and `FPU_INDEX`, and extend the value stack so a slot can hold either a `PyObject*` or a tag that names a register inside the unit. A guard checks that the operands are still doubles. A cache miss or a failed guard falls back to the normal path.

6.2 Performance bound

The unit covers boxing (0.353 G), sequence element access (0.359 G), operator dispatch (0.293 G) and index conversion (0.148 G). Together that is 1.153 G of the baseline's 2.909 G instructions per call, or 39.6% of the measured profile, and 41.1% once debug-build-only code is taken out. That gives an Amdahl ceiling of 1.70× release-equivalent. It would not reach that, because the guards and the tag handling are not free.

If the unit eliminates	Runtime cut	Speedup
60% of addressable work	24.7%	1.33×
70%	28.8%	1.40×
80%	32.9%	1.49×
90%	37.0%	1.59×

6.3 Trade-offs and failure modes

- Area- a 1024-entry site cache at 8 bytes per entry is 8 KB, the register file is 32 doubles or 256 bytes, plus the guard and bounds-check logic. Small next to an L1 cache, and shared by every numeric loop rather than repeated.
- Power- active only while the new opcodes are issuing, and idle on everything else.
- Energy- the same 5.4 M floating-point operations finish in far fewer cycles, so energy per operation goes down even if the instantaneous power goes up while the unit is running.
- Failure modes- a site that sees several types thrashes the cache and pays the guard on every miss. A value that escapes straight away gets boxed anyway, so the boxing is delayed rather than avoided. The cache has to be invalidated when a type changes. nbody runs the same five bodies

for 20,000 timesteps, so it is close to the best case for this design, and we say so rather than claiming it generalises.

- The opt2 constraint- the issue path must cost less than the work it replaces. FPU_OP must be a real opcode in the dispatch loop, not a C function call.

7 Conclusions

7.1 nbody

nbody spends 11.1% of its runtime on the floating-point arithmetic the algorithm needs, and 88.9% on the cost of doing that arithmetic in a bytecode interpreter on heap objects. CPython runs 539 instructions per floating-point operation; compiled C runs 3.66.

opt1 met the 7% threshold at -10.3%, and the three mechanisms in its attribution table explain 80% of the measured difference. opt2 taught us the most. The arithmetic did get cheaper: the whole libm pow path disappears from the profile and the bucket falls by 0.059 G, which is two thirds of the budget the profile said was there. The program still got 3.7% slower than opt1, because a Python function call, the math module's dispatch, a global lookup and two extra boxed temporaries cost 0.166 G to express it. Collecting most of a ceiling is not the same as winning. opt3 shows what removing the interpretation completely gives: 147× fewer instructions, and a second measurement correction, because at 3.82 ms per call the startup cost we had been able to ignore became 37% of the measurement.

7.2 pyflate

pyflate is a data-movement workload. It makes 12× more cache references per instruction than nbody, misses 2.8× more often, and takes 127,912 page faults per fixed-work run, which come from object churn rather than from the data being decompressed. opt1 removed 72% of the move-to-front cost by replacing a five-allocation list rebuild with two memmoves, giving -12.7%. opt2 removed another 0.74 G instructions per call, and the attribution showed it worked for a different reason than we predicted: 92% of the gain came from removing an interpreted search loop, not from faster dictionary lookups.

We had to make two corrections that came from checking rather than assuming, and we report both rather than hiding them. 18.3% of this benchmark's profile is debug-allocator code that a release interpreter does not have. And our first attribution of the list cost to the DEFLATE back-reference copy was wrong, because the DEFLATE path never runs for a bzip2 input.

7.3 The hardware finding

We implemented the move-to-front unit in Verilog and verified it against 89,837 operations captured from a real decode. It reproduces pyflate's symbol stream exactly. It replaces about 21,400 executed instructions per operation with one cycle, using a 256×8 -bit table in flip-flops, a conditional parallel shift and a two-state control FSM, at an estimated 200 MHz and about 2 K flip-flops of storage.

For nbody, only 11% of the runtime is the arithmetic the algorithm actually needs. The interpreter spends 539 instructions per floating-point operation where compiled C spends 3.66.

For pyflate, an accelerator that makes its target 1,294× faster still only makes the benchmark 1.23× faster.

Together, these results show why profiling comes before acceleration: the useful result is not only how fast a target can run, but how much of the total runtime that target can remove.

Appendix Repository and reproduction

All code, measurements and reports: https://github.com/imalmogit/hwswProject_repo.git

Layout

Path	Contents
report_nbody.txt	full nbody analysis, optimizations, results, HW proposal
report_pyflate.txt	full pyflate analysis, optimizations, results, HW proposal
script_nbody.sh	complete nbody pipeline
script_pyflate.sh	complete pyflate pipeline and RTL verification
prompt.txt	AI prompts used during the project, and where they were wrong
hardware/	mtf_unit.v, tb_mtf_unit.v, gen_stimulus.py, Makefile, README.md
benchmarks/	the five custom benchmark variants
scripts/	measurement and verification harness
results/	measurement outputs by variant- every figure in this document
docs/	measurement method note
env/	QEMU guest environment audit

Running it

```
./script_nbody.sh      # setup, verify, baseline, optimized, profile, compare
./script_pyflate.sh    # the same, plus the RTL verification
cd hardware && make     # hardware verification only
```

Correctness is checked before any measurement: nbody results are compared against the original final energy after 20,000 timesteps, and pyflate results against the benchmark's MD5 digest. nbody uses -l 1 -w 1 -n 100 and pyflate uses -l 1 -w 1 -n 20, including their baselines. Do not change those values unless you rerun every variant.

Limitations

- Measurements use a debug (--with-pydebug) interpreter and a single-vCPU QEMU guest. Every bottleneck figure carries a release-equivalent correction, but the correction is an estimate.
- The accelerator is verified in simulation only. We did not synthesize it, so the area and frequency numbers are our own estimates rather than tool reports.
- The instructions-per-mechanism figures apply a share of samples to a total instruction count, which assumes IPC is roughly the same across mechanisms.